

Fixing Mutual Authentication

Encryption does not ensure authenticity: use MAC to authenticate messages

Prevent redirection: identify party in challenge

Better to use separate keys for each direction

Prevent replay and reorder

- Identify flow and connection
- Prevent use of old challenge: randomness, time, or state

Do not provide the adversary with oracle access! (like in SNA)

- Do not compute values from Adversary
- Include self-chosen nonce in the protected reply

Secure Two-Party Handshake Protocol (2PP)

Use MAC rather than encryption to auth

Prevent redirection: include identities (A, B)

Prevent replay and reorder:

- Nonces (N_A, N_B)
- Separate 2nd and 3rd flows: 3 vs 2 input blocks

Provably secure, since MACs are deterministic

New nonces are generated for each flow, prevents concurrent attack

Flow numbers ensure prevention of redirection

Authenticated Request-Control Protocols

Besides authenticating entities, these protocols auth the exchange of a request and a response between the entities.

Required properties:

- Request Authentication: the request was indeed sent by the sender.
- Response Auth: the response was indeed sent by the receiver (to which the request was intended for)
- No Replay

Five variants:

- 2PP-RR (two party protocol, request-response)
- 2RT-2PP (2 round trip) (a round trip is two flows)
- Counter-based-RR
- Time-based RR
- Key-exchange (session key)

2PP-RR

A three-flow nonce-based protocol.

Significant drawback:

- The request is sent by the responder and the initiator sends the response.
- So, the initiator must wait for a request rather than sending it

2RT-PP

A four-flow nonce-based protocol.

Mainly fixes the drawback of 2PP-RR

Counter-Based Authenticated RR

Simple stateful (counter) solution, requiring only one round:

- Unidirectional (if bidirectional is needed, a separate instance of the protocol for each direction needs to be executed)
- Parties maintain synchronized counter i of requests (and responses) to avoid replay attacks
- Recipient validates if counter received is $i + 1$
- Both parties must remember the counter

Counters are only initialized once

Time-Based Auth RR

Simple stateful (time) solution, requiring only one round:

- Use local clocks T_A, T_B instead of counters with two assumptions: bounded delays and bounded clock skews
- Responder (Bob):
 - Rejects request if $T_B > T_A + \Delta$ OR if he received a larger T_A already (where $\Delta = \Delta_{skew} + \Delta_{delay}$)
 - Maintains last T_A received until $T_A + \Delta$
- Initiator does not need any state, when can receiver discard this?

Don't reuse keys across parties (TWO party protocol)

2RT-2PP with Confidentiality

Secure connection: authentication, freshness, and secrecy

- Independent keys: for encryption $k.e$, for auth $k.a$
- How can we derive them both from a single key k ? The PRF from before: $k.e = PRF_k("Encrypt")$, $k.a = PRF_k("MAC")$

Improve security by changing keys between sessions?

2PP Key Exchange Protocol

Allows generating independent session keys (i.e. each session will have its own key denoted as k_i^S)

These keys are derived from the sharedkey that we now call a long-term 'master key' denoted as k^M